LVGL开发准备

LVGL开发准备
- LVGL 简介
  - LVGL 的核心优势
- lv_binding_micropython
  - 核心特性
  - 适用场景
  - 使用限制
  - 优势
- 如何在K230上学习 lvgl
  - 快速开始

本节教程是后续LVGL开发的必要基础

> 虽然没有选配屏幕也可以学习lvgl的写法，但是很多触摸相关的功能用不了，学习体验会较差

LVGL 简介

LVGL (Light and Versatile Graphics Library) 是一个免费、开源的图形库，专门为嵌入式系统设计。

如果你想要在K230上绘制简单的线条、图形，你可以使用 draw_xxxx 系列的图形处理方法，详情可以参考前面的教程。而如果你想要在K230上显示一些较为复杂、并且能与用户交互的图形（比如说文本框、按钮、进度条等）

那么我们一般会选择一款GUI（Graphical User Interface,图形用户界面）库来实现。而LVGL就是一个非常好用的GUI库

LVGL 的核心优势

资源占用低

- 最小内存要求仅需64KB ROM和16KB RAM
- 适合资源受限的MCU平台
- 组件可按需引入，优化资源占用

跨平台兼容性

- 支持几乎所有主流MCU平台
- 可移植性强，硬件抽象层设计良好
- 支持多种操作系统：FreeRTOS、RT-Thread等

开发效率

- C语言开发，学习曲线平缓（也支持其他语言，比如Micropython，后文我们会提到）
- API设计简洁直观
- 丰富的示例代码和文档支持

lv_binding_micropython

> Github 链接 lvgl/lv_binding_micropython: LVGL binding for MicroPython
>
> 注：Github 经常访问不了，属于正常情况，不看这个链接的内容不会影响接下来的学习

前文中我们讲了一下LVGL。原生的LVGL实际上是一个C语言项目，并不能使用Micropython进行编程。随着使用Micropython编程的嵌入式场景越来越多，于是诞生了lv_binding_micropython这个项目。

lv_binding_micropython 是一个将 LVGL 绑定到 MicroPython 的开源项目，让开发者能够使用 Python 语言开发 LVGL 图形界面。以下是主要特点：

核心特性

1. **易用性**

- 使用 Python 语法开发 LVGL
- 降低了 LVGL 的使用门槛
- 支持快速原型开发

2. **功能完整**

```
# 示例代码
# 注意，这个是伪代码，不能直接在K230上运行
import lvgl as lv
scr = lv.obj()  # 创建屏幕对象
btn = lv.btn(scr)  # 创建按钮
btn.center()  # 居中按钮
label = lv.label(btn)  # 创建标签
label.set_text("Button")  # 设置文本
```

适用场景

- 快速原型验证
- 教育教学
- 简单的图形界面开发
- 非性能关键型应用

使用限制

1. **性能开销**

- 相比C语言版本有一定性能损失
- 内存占用相对更大

2. **硬件要求**

- 需要更大的Flash和RAM
- 建议使用主频超过100MHz的MCU

优势

- 开发效率高
- 调试方便
- 代码量减少
- 适合快速验证

如何在K230上学习 lvgl

LVGL是一个相对复杂的组件库，我们的教程很难涉及到所有的内容，所以阅读官方的文档就变得很有必要了

K230固件中的lvgl库使用的是v8.3版本

官方文档链接如下：Welcome to the documentation of LVGL! — LVGL documentation

> 如果链接失效了，可以到我们的开发板资料里找到LVGL.pdf，这是一个离线版的文档备份

官方的教程是以C语言为主的，其中虽然有Micropython写的部分，但是**文档内的写法和实际调用存在不一致的情况**，比如函数名称差异、调用方法差异等。为解决这个问题，我把K230固件内中LVGL真实的常量写法、函数名称全部打印出来了，放在开发板资料中。大家需要结合 **官方文档** 和 **实际支持调用的写法**（0.开发板资料 / 其它第三方文档 / lvgl_.txt） 这两个文档，进行开发学习。

快速开始

完整代码文件【源码汇总 / 12.Lvgl / 01.start.py】

下面是一段lvgl代码的基本结构示例

```
import lvgl as lv
from media.display import *
import time  # 显式导入time模块 / Explicitly import time module

# 显示屏分辨率配置 / Display resolution configuration
DISPLAY_WIDTH = 640   # 显示屏宽度 / Display width
DISPLAY_HEIGHT = 480  # 显示屏高度 / Display height

def disp_drv_flush_cb(disp_drv, area, color):
    """
    显示驱动刷新回调函数
    Display driver flush callback function

    Args:
        disp_drv: 显示驱动对象 / Display driver object
        area: 刷新区域 / Refresh area
        color: 颜色数据 / Color data
    """
    global disp_img1
    Display.show_image(disp_img1)  # 显示图像缓冲区 / Show image buffer
    time.sleep(0.01)  # 适当延时确保显示稳定 / Small delay to ensure stable display
    disp_drv.flush_ready()  # 通知LVGL刷新完成 / Notify LVGL that flush is complete

def display_init():
    """
    显示设备初始化函数
    Display device initialization function
    """
    # 初始化ST7701显示屏 / Initialize ST7701 display
    Display.init(
        Display.ST7701,
        width=DISPLAY_WIDTH,
        height=DISPLAY_HEIGHT,
        to_ide=True  # 启用IDE显示功能 / Enable IDE display function
    )

    # 初始化媒体管理器 / Initialize media manager
    MediaManager.init()

def lvgl_init():
    """
```

```
    LVGL初始化函数
    LVGL initialization function
    """
    global disp_img1

    # 初始化LVGL库 / Initialize LVGL library
    lv.init()

    # 创建显示缓冲区 / Create display buffer
    disp_img1 = image.Image(
        DISPLAY_WIDTH,
        DISPLAY_HEIGHT,
        image.BGRA8888  # 使用BGRA8888颜色格式 / Use BGRA8888 color format
    )

    # 创建显示驱动 / Create display driver
    disp_drv = lv.disp_create(DISPLAY_WIDTH, DISPLAY_HEIGHT)

    # 设置显示缓冲区 / Set display buffers
    disp_drv.set_draw_buffers(
        disp_img1.bytearray(),
        None,  # 单缓冲模式 / Single buffer mode
        disp_img1.size(),
        lv.DISP_RENDER_MODE.DIRECT  # 直接渲染模式 / Direct rendering mode
    )

    # 设置刷新回调函数 / Set flush callback
    disp_drv.set_flush_cb(disp_drv_flush_cb)

def display_deinit():
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(50)
    # deinit display
    Display.deinit()
    # release media buffer
    MediaManager.deinit()

def lvgl_deinit():
    global disp_img1
    lv.deinit()
    del disp_img1

def main():
    """
    主函数
    Main function
    """
    try:
        # 初始化显示设备和LVGL / Initialize display device and LVGL
        display_init()
        lvgl_init()
        print("LVGL initialization completed")

        # LVGL主循环 / LVGL main loop
        while True:
            # 运行LVGL定时器处理程序 / Run LVGL timer handler
            period = lv.timer_handler_run_in_period(1)
            time.sleep_ms(period)
```

```
    except Exception as e:
        display_deinit()
        lvgl_deinit()
        print(f"Error occurred: {e}")

if __name__ == "__main__":
    main()
```

基本结构包含了下面几个主要部分：

显示设备初始化：

- 初始化ST7701显示屏
- 设置显示分辨率
- 初始化媒体管理器

LVGL初始化：

- 初始化LVGL库
- 创建和配置显示缓冲区
- 设置显示驱动和回调函数

主循环：

- 运行LVGL定时器处理程序
- 错误检查与处理

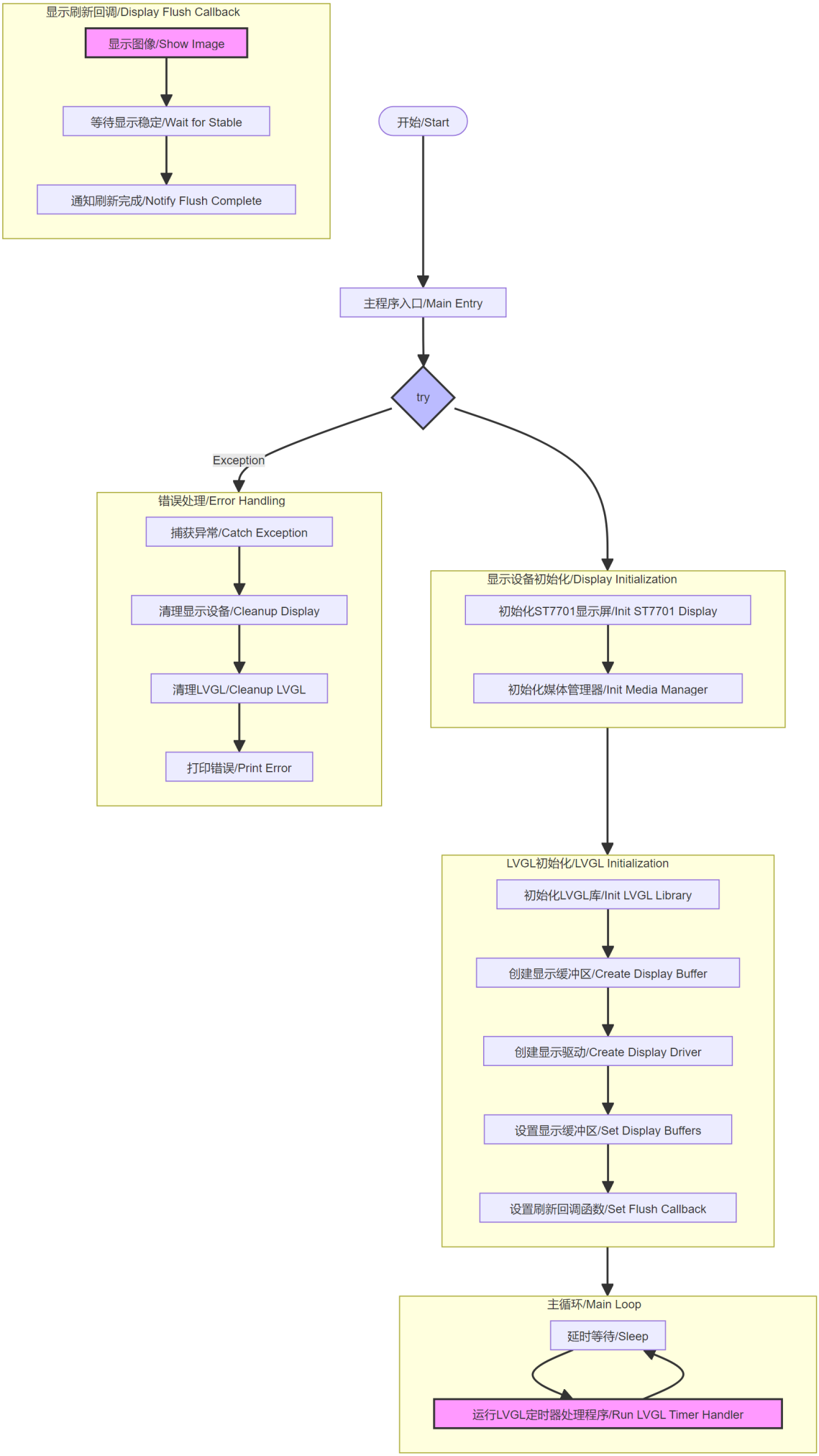
显示刷新回调：

- 显示图像
- 等待显示稳定
- 通知刷新完成

清理流程：

- 显示设备清理
- 媒体管理器清理
- LVGL清理
- 缓冲区删除

这几个过程是在K230上使用lvgl必不可少的部分，虽然代码看起来很多，但是基本上不需要做什么修改，每次写新的LVGL程序的时候直接复制即可


显示刷新回调/Display Flush Callback
显示图像/Show Image
等待显示稳定/Wait for Stable
通知刷新完成/Notify Flush Complete
开始/Start
主程序入口/Main Entry
try
Exception
错误处理/Error Handling
捕获异常/Catch Exception
清理显示设备/Cleanup Display
清理LVGL/Cleanup LVGL
打印错误/Print Error
显示设备初始化/Display Initialization
初始化ST7701显示屏/Init ST7701 Display
初始化媒体管理器/Init Media Manager
LVGL初始化/LVGL Initialization
初始化LVGL库/Init LVGL Library
创建显示缓冲区/Create Display Buffer
创建显示驱动/Create Display Driver
设置显示缓冲区/Set Display Buffers
设置刷新回调函数/Set Flush Callback
主循环/Main Loop
延时等待/Sleep
运行LVGL定时器处理程序/Run LVGL Timer Handler

我们运行这段代码示例，可以看到K230的屏幕上出现了白色的背景且没有出现报错信息，这样就说明lvgl初始化成功了

接下来的章节里，我们将介绍几个LVGL常用的组件